

**Smt. Chandaben Mohanbhai Patel Institute of Computer
Applications, CHARUSAT**

BCA Semester-III

CAUE302 : Python Programming

**Week 3: Implementation of Control Flow Structures (Conditional &
Iterative Statements)**

- **Introduction to Control Flow**

- Control Flow Structures break this linear execution path, enabling a program to dynamically branch out or repeat code blocks based on runtime values. This structural logic forms the baseline for application development in Python programming.

- **Part A: Conditional Statements (Decision Making)**

- Conditional structures evaluate a specific logical expression. If the condition evaluates to True, a dedicated block of code runs; if it evaluates to False, that block is completely bypassed.

- **The Indentation Rule in Python**

- Unlike traditional programming languages (like C, C++, or Java) that rely on curly braces { } to define functional blocks, Python dictates structural code blocks using Indentation (whitespace layout alignment). A colon (:) initiates a new block context, and all subsequent lines within that specific block must share an identical tab or space indentation alignment.

- **Topic 1: The Simple if Statement**

What is an if statement?

- The if statement is the simplest decision-making structure. It is used when an action needs to be executed only if a specific condition evaluates to True. If the target condition is False, the code inside the block is skipped completely.

Syntax Structure:

```
if condition:  
    # Code block to execute if condition is True  
    statement_1  
    statement_2
```

- **Example 1: Passing Marks Verification**

```
# Check if a student's marks qualify as passing
```

```
marks = int(input("Enter your marks: "))
```

```
if marks >= 40:
```

```
    print("Result Status: PASSED")
```

```
    print("Congratulations! Keep up the good work.")
```

- **Example 2: Discount Eligibility**

```
# Check if a customer qualifies for a shopping discount
```

```
bill_amount = float(input("Enter total shopping bill amount: "))
```

```
if bill_amount > 1000:
```

```
    print("Eligibility: You have won a 10% discount coupon!")
```

- **Example 3: Age Restriction Check**

```
# Check if a user is old enough to apply for a driving license
```

```
age = int(input("Enter your age: "))
```

```
if age >= 18:
```

```
    print("Status: Eligible to apply for a standard license.")
```

- **Topic 2: The Dual-Branch if-else Statement**

What is an if-else statement?

- The if-else statement provides an alternative path of execution when the if condition check fails. It establishes a strict "Either / Or" decision structure where exactly one of the two blocks will execute.

Syntax Structure:

if condition:

```
# Executes if condition is True
statement_A
```

else:

```
# Executes if condition is False
statement_B
```

Complete Code Example 1: Even or Odd Number Classifier

```
# Determine if an input integer is even or odd
number = int(input("Enter any integer number: "))
if number % 2 == 0:
    print("The value is an EVEN number.")
else:
    print("The value is an ODD number.")
```

Complete Code Example 2: Authentication System

```
# Simulate a simple password validation check
entered_password = input("Enter password: ")
secret_key = "python123"

if entered_password == secret_key:
    print("Access Granted!")
    print("Welcome to your dashboard.")
else:
    print("Access Denied!")
    print("Incorrect password. Please try again.")
```

Complete Code Example 3: Temperature Category

```
# Check whether it is hot or cold outside
temp = float(input("Enter current temperature in Celsius: "))
if temp >= 30:
    print("Weather Alert: It is hot outside!")
else:
    print("Weather Alert: The weather is pleasant or cold.")
```

Topic 3: The Multi-Branch if-elif-else Chain

What is an if-elif-else statement?

- When you have more than two possibilities, you use an if-elif-else chain. The program checks conditions from top to bottom.
- As soon as it runs into its first True condition, it executes that specific block and drops completely out of the remaining chain. The terminal else serves as a fallback catch-all.

Syntax Structure:

```
if condition_1:
    statement_1
elif condition_2:
    statement_2
elif condition_3:
    statement_3
else:
    fallback_statement
```

Complete Code Example 1: Academic Grading Portal

```
# Convert numeric scores into an official Grade Letter
score = int(input("Enter your exam score out of 100: "))
if score >= 70:
    print("Assigned Grade: A")
elif score >= 50:
    print("Assigned Grade: B")
elif score >= 40:
    print("Assigned Grade: C")
```

```
else:  
    print("Assigned Grade: F (Fail)")
```

Complete Code Example 2: Cinema Ticket Price Coordinator

```
# Calculate movie ticket prices based on customer age group  
viewer_age = int(input("Enter your age: "))  
  
if viewer_age < 5:  
    print("Ticket Price: Free")  
elif viewer_age <= 12:  
    print("Ticket Price: INR 150 (Child Rate)")  
elif viewer_age <= 60:  
    print("Ticket Price: INR 300 (Standard Adult Rate)")  
else:  
    print("Ticket Price: INR 200 (Senior Citizen Special Rate)")
```

Complete Code Example 3: Arithmetic Menu Selection

```
# Perform specific operations based on menu selection  
print("--- Simple Menu ---")  
print("1. Add")  
print("2. Subtract")  
print("3. Multiply")  
choice = int(input("Enter choice (1-3): "))  
  
num1 = 10  
num2 = 5  
  
if choice == 1:  
    print("Result:", num1 + num2)  
elif choice == 2:  
    print("Result:", num1 - num2)  
elif choice == 3:  
    print("Result:", num1 * num2)  
else:  
    print("Invalid Selection!")
```

- **Part B: Iterative Statements (Loops)**

Iterative statements execute a specific target code block multiple times, automating redundant programming steps and handling repetitive datasets smoothly.

Topic 4: The Condition-Driven while Loop

What is a while loop?

- A while loop repeats a designated block of code as long as a chosen control condition evaluates to True. It is highly important to include an increment or decrement step inside the loop body to modify the tracking variable so the loop eventually terminates.

Syntax Structure:

```
while condition:
    statements
    increment_or_decrement_step
```

Example 1: Basic Sequential Number Counter

```
# Print a simple numeric series from 1 up to 5
counter = 1
while counter <= 5:
    print("Current Counter Value:", counter)
    counter = counter + 1 # Incremental modification step
print("While iteration complete.")
```

Example 2: Interactive Login Limit Simulator

```
# Simulate a countdown pattern for failed retry limits
attempts = 3
while attempts > 0:
    print("Warning: Secure connection active. Remaining attempts:", attempts)
    attempts = attempts - 1 # Decremental step
print("System Locked out. Reset required.")
```

Topic 5: The Sequence-Driven for Loop & range() Function

What is a for loop?

The for loop is used to iterate over items of any sequence (such as a string, list, or generated numeric sequence). In Python, it is commonly paired with the built-in range() function.

- range(stop): Generates a sequence from 0 up to stop - 1.
- range(start, stop): Generates integers from start up to stop - 1.
- range(start, stop, step): Generates integers from start up to stop - 1, incrementing by the step value.

Syntax Structure:

```
for variable in range(start, stop, step):  
    statements
```

Example 1: Even Number Series Generator

```
# Print all even numbers between 2 and 20 using step sequencing  
print("Generating Even Numbers:")  
for val in range(2, 21, 2):  
    print(val, end=" ")  
print()
```

Example 2: String Character Traverser

```
# Traverse and display every individual letter of a word string  
word = "PYTHON"  
print("Traversing letters:")  
for letter in word:  
    print("Letter token:", letter)
```

• Topic 6: Loop Control Modifiers (break & continue)

What are break and continue?

Loop control modifiers alter the operational lifecycle of loops dynamically based on a condition matching evaluation:

- **break:** Instantly terminates the current loop execution entirely, bypassing any remaining loops and transferring flow to the next lines of the script.
- **continue:** Skips the remaining lines of code in the current iteration step and jumps immediately back up to the loop verification header for the next turn.

Syntax Structure:

```
# Inside a loop block:
if break_condition:
    break
```

```
if skip_condition:
    continue
```

Example 1: Modifier Filtering Operations

```
print("--- Demonstrating CONTINUE (Skipping value 3) ---")
for val in range(1, 6):
    if val == 3:
        continue # Aborts print for 3
    print("Value:", val)
```

```
print("\n--- Demonstrating BREAK (Stopping early at 3) ---")
for val in range(1, 6):
    if val == 3:
        break # Completely breaks out of loop execution
    print("Value:", val)
```

Example 2: Target Number Searching

```
# Search for a specific target value and stop loop instantly when found
target = 7
for num in range(1, 11):
    print("Checking item:", num)
    if num == target:
        print(f"Success: Target {target} found! Stopping search.")
        break
```